
PERPETUAL MOTION SQUAD
POCKETEXPENSE+

Installation Guide

For end users and developers

Purvansh Sahu Yash Arya Aman Anand

PROJECT	PocketExpense+ — Offline-First Financial Tracking System
REPOSITORY	github.com/purvanshh/PocketExpense-
TEAM	Perpetual Motion Squad
INSTITUTION	BITS Pilani
PROJECT GUIDE	Mr. Murali Raghu
DATE	August 2026

Contents

1. Before You Begin
2. End-User Installation
3. Developer Installation
4. Environment Configuration
5. Running the Test Suites
6. Common Problems
7. Docker
8. Deployment
9. Quick Start Checklist

1. Before You Begin

1.1 Which path do you need?

You want to...	Go to
Install the app on a phone and use it	§2 End-User Installation
Run the code, modify it, or develop against it	§3 Developer Installation
Deploy the backend to a server	§8 Deployment

1.2 How PocketExpense+ is distributed

Important: PocketExpense+ is *not published on the Google Play Store or the Apple App Store*. It is distributed directly by the developer — run it in **Expo Go** (QR code) or install a **full build** (EAS-generated installation link).

1.3 A note on native features

Feature	Requires
SMS transaction detection	A development or production build (Android)
On-device receipt OCR	A development or production build

Everything else — camera, notifications, export, and the entire offline architecture — works in Expo Go. When a development build is absent, receipt scanning degrades to attaching the photograph and entering the amount manually. This is a property of Expo's managed workflow: Expo Go ships a fixed set of native modules, and modules outside that set require a custom build.

2. End-User Installation

2.1 Run it in Expo Go (quickest for testing)

Install **Expo Go** (free in the Play Store / App Store). **Android:** open Expo Go → **Scan QR code** → point at the developer's QR code. **iOS:** open the Camera app → point at the QR code → tap the banner.

SMS detection will not work through Expo Go — use a full build for the complete feature set.

2.2 Install a full build (all features)

Open the installation link the developer sent you. On **Android**, allow "Install unknown apps" when prompted. On **iOS**, trust the developer under Settings → General → VPN & Device Management if asked. SMS detection is unavailable on iOS — Apple provides no API for reading SMS content.

2.3 First launch

Open the app → **Don't have an account?** → enter name, email, and a password of at least 6 characters → **Create Account**. See the User Manual for what to do next.

3. Developer Installation

3.1 Prerequisites

Requirement	Minimum	Check with	Get it from
Node.js	20.0	<code>node --version</code>	nodejs.org
npm	10.0	<code>npm --version</code>	Bundled with Node
Git	2.30	<code>git --version</code>	git-scm.com
MongoDB	6.0	<code>mongod --version</code>	mongodb.com — or Atlas
Expo Go	Latest	—	Play Store / App Store

Optional (native builds): Android Studio · Xcode 15+ (macOS only) · EAS CLI (`npm install -g eas-cli`).

3.2 Clone and install

```
git clone https://github.com/purvanshh/PocketExpense-.git
cd PocketExpense-
cd server && npm install && cp .env.example .env # backend
cd .. && npm install # frontend
```

3.3 Backend setup

Set two values in `server/.env` that have no safe default:

```
MONGODB_URI=mongodb://localhost:27017/pocketexpense
JWT_SECRET=<32+ random characters> # generate: node -e "console.log(require('crypto').randomBytes(32)).toString('hex')"
```

Do not use the placeholder from `.env.example` outside local development — anyone knowing the secret can mint valid tokens.

Start MongoDB: macOS `brew services start mongodb-community` · Linux `sudo systemctl start mongod` · Windows `net start MongoDB`, or use Atlas and paste the connection string into `MONGODB_URI`.

Run and verify:

```
npm run dev # server starts on http://0.0.0.0:5001 with cron service
curl http://localhost:5001/api/health
→ { "success": true, "message": "PocketExpense+ API is running", "data": { "status": "ok", "environment": "development" } }
```

3.4 Frontend setup

Point the app at your backend. `localhost` will not work from a phone (the phone's `localhost` is the phone itself) — find your machine's LAN address (`ipconfig getifaddr en0` on macOS / `hostname -I` or `ipconfig` on Windows) and set it in `src/store/api/apiClient.ts`:

```
const YOUR_IP = '192.168.1.42'; // your machine's address
const PORT = '5001';
```

Both devices must be on the same network — a phone on mobile data cannot reach your home Wi-Fi server.

Start and open:

```
npx expo start    # Android: Expo Go → Scan QR · iOS: Camera → banner · a / i / w for emulator/simulator/
```

3.5 Building for native features

Required for SMS detection and on-device OCR.

```
# Cloud build (recommended)
npm install -g eas-cli && eas login
eas build --platform android --profile development
npx expo start --dev-client

# Local build
npx expo run:android  # Android Studio required
npx expo run:ios      # Xcode, macOS only
```

4. Environment Configuration

4.1 Backend variables

All configuration lives in `server/.env`: `PORT` (5001) · `HOST` (0.0.0.0) · `NODE_ENV` (development) · `MONGODB_URI` (**required**) · `JWT_SECRET` (**required**, ≥32 chars) · `JWT_EXPIRE` (7d) · `RATE_LIMIT_WINDOW_MS` (900000) · `RATE_LIMIT_MAX` (100) · `CORS_ORIGIN` (* — restrict in production).

4.2 Development .env

```
PORT=5001 HOST=0.0.0.0 NODE_ENV=development
MONGODB_URI=mongodb://localhost:27017/pocketexpense
JWT_SECRET=a1b2c3d4e5f6... # 32+ characters
JWT_EXPIRE=7d
RATE_LIMIT_WINDOW_MS=900000 RATE_LIMIT_MAX=100
CORS_ORIGIN=*
```

4.3 Production .env

Key differences: `NODE_ENV=production` (hides stack traces), an Atlas connection string (managed backups), a strong `JWT_SECRET` (the placeholder is public), and an explicit `CORS_ORIGIN` (wildcard permits any site to call your API).

```
PORT=5001 HOST=0.0.0.0 NODE_ENV=production
MONGODB_URI=mongodb+srv://user:pass@cluster.mongodb.net/pocketexpense
JWT_SECRET=<64 random hex, never committed> JWT_EXPIRE=7d
RATE_LIMIT_WINDOW_MS=900000 RATE_LIMIT_MAX=100
CORS_ORIGIN=https://your-domain.example
```

4.4 Keeping secrets out of version control

`.env` is in `.gitignore` — confirm with `git check-ignore server/.env`. If it was ever committed, rotate the secret (removing the file does not remove it from history).

5. Running the Test Suites

5.1 Backend — 96 tests

```
cd server
npm test          # all 96   (6 suites, in-memory MongoDB)
npm run test:unit  # 48      npm run test:integration # 17
npm run test:edge  # 31      npm run test:coverage   # ~82%
```

5.2 Frontend — 102 tests

```
npm test          # all 102 (6 suites)
npm run test:sms   # SMS parser only (42) — prints 99% accuracy report
```

5.3 Static checks & load testing

```
npx tsc --noEmit && npx expo lint  # 0 errors
cd server && npm install -g artillery
npm start &                          # terminal 1
artillery run tests/performance/artillery.yml # enforces p95 < 500 ms, p99 < 1000 ms
```

6. Common Problems

6.1 Backend

Problem	Fix
MongoNetworkError: ECONNREFUSED	Start MongoDB: <code>brew services start mongodb-community</code> (or <code>systemd/net</code>)
MongoServerError: bad auth	Re-check Atlas credentials; URL-encode special characters
EADDRINUSE :::5001	<code>lsof -ti:5001 xargs kill -9</code> , or change <code>PORT</code>
secretOrPrivateKey must have a value	Set <code>JWT_SECRET</code> in <code>.env</code> and restart
Cannot find module 'express'	<code>cd server && npm install</code>
Atlas connection times out	Atlas → Network Access → add your IP

6.2 Frontend

Problem	Fix
Network request failed	Re-check <code>YOUR_IP</code> ; ensure both devices share a network
QR code will not scan	Try <code>npx expo start --tunnel</code>
Unable to resolve module	<code>npx expo start --clear</code>
Metro fails to start	<code>rm -rf node_modules package-lock.json && npm install</code>
App opens then closes	Check the terminal output for the stack trace

Changes not appearing	Shake the device → Reload
-----------------------	---------------------------

6.3 SMS detection & OCR

Problem	Fix
Toggle does nothing in Expo Go	Requires a development build — see §3.5
Permission dialog never appears	Phone Settings → Apps → PocketExpense+ → Permissions → SMS
No text extracted from a receipt	Expected in Expo Go — photo is still attached; enter the amount manually
Camera will not open	Phone Settings → Apps → PocketExpense+ → Permissions → Camera

6.4 Tests

Problem	Fix
Backend tests hang	Scripts already pass <code>--forceExit</code> ; use <code>npm test</code>
mongodb-memory-server download fails	Run once with a connection; the binary is then cached
Frontend tests fail resolving a module	Test only dependency-light <code>.ts</code> modules

7. Docker

The backend ships with a `Dockerfile` based on `node:20-alpine`.

7.1 Build and run

```
cd server
docker build -t pocketexpense-api .
docker run -p 5001:5001 --env-file .env pocketexpense-api
```

7.2 With MongoDB via Compose

Create `docker-compose.yml` in `server/` with an `api` service (build `.`, port `5001:5001`, env from `.env`, `MONGODB_URI=mongodb://mongo:27017/pocketexpense`, depends_on `mongo`) and a `mongo` service (image `mongo:7`, volume `mongo-data:/data/db`):

```
JWT_SECRET=$(node -e "console.log(require('crypto').randomBytes(32).toString('hex'))" ) \
  docker compose up -d
docker compose logs -f api      # follow logs
docker compose restart api      # restart
docker compose down [-v]       # stop [and delete data]
```

8. Deployment

8.1 Backend — Render.com

The repository includes `server/render.yaml`. Push to GitHub → in Render choose New → Web Service → connect the repo → set the root directory to `server` → add the §4.3 environment variables → deploy. Health check path: `/api/health`; build `npm install`; start `npm start`.

8.2 Backend — anywhere else

The application is a standard Node process — it runs on Railway, Fly.io, Heroku, a VPS, or any container host. Requirements: Node.js 20+, a reachable MongoDB instance, the §4.3 variables, and a process manager.

Caveat: the recurring scheduler runs in-process without a distributed lock — run a single instance, or two instances may both generate the same recurring transaction.

8.3 Database — MongoDB Atlas

Create a free M0 cluster, add a read/write user, allowlist your server's IP, and paste the connection string into `MONGODB_URI`.

8.4 Frontend — EAS Build

Profiles in `eas.json`: `development` (dev client) · `preview` (internal testers) · `production` (store-ready). Build: `eas build --profile <profile> --platform android`.

Before a production build, point the client at your deployed API rather than a LAN address.

8.5 Publishing to stores (future work)

PocketExpense+ is *not published* on any app store. If publishing is undertaken, prepare the icon, screenshots, description, and — because the app requests SMS access on Android — a privacy policy, then use `eas submit`.

9. Quick Start Checklist

9.1 Developer setup

Check	Done
Node.js 20+ · MongoDB running (or Atlas) · repo cloned	☐
<code>cd server && npm install · cp .env.example .env · set MONGODB_URI + JWT_SECRET</code>	☐
<code>npm run dev</code> starts · <code>curl /api/health</code> → success: true	☐
<code>cd .. && npm install · YOUR_IP</code> set in <code>apiClient.ts</code>	☐
<code>npx expo start</code> → QR → app opens in Expo Go · account created	☐

9.2 Verification and pre-deploy

Check	Expected
<code>npx tsc --noEmit / npx expo lint</code>	0 errors
<code>npm test / npm test --prefix server</code>	102 / 96 passed
<code>npm run test:coverage --prefix server</code>	~82%
Production env: strong <code>JWT_SECRET</code> , restricted <code>CORS_ORIGIN</code> , Atlas <code>MONGODB_URI</code> , <code>.env</code> out of git, single cron instance	☐
Client API URL points at deployed backend; health check reachable	☐

— End of Installation Guide —